

# GERMEN 02

Umbrales y Arquitectura Técnica del Sistema Capaz

Aplicación en IA local para logística de alimentos

Ignacio López Villanueva

A Coruña, 2026

# Propósito

GERMEN 02 fija los umbrales operativos de los cinco invariantes del sistema capaz y los traduce en parámetros reales que una IA local puede medir para blindar la logística del puesto de alimentos. Incluye pseudocódigo, evaluación de umbrales y el ciclo capaz completo implementable en sistemas locales sin dependencia de red.

## 1 Umbrales operativos del sistema capaz

Los cinco invariantes de GERMEN 01 se traducen aquí en parámetros medibles por una IA local. Cada invariante se clasifica en tres estados: seguro, riesgo y colapso.

### 1.1 Energía

- **Seguro:** CPU < 70%, RAM < 70%, Temp < 65°C
- **Riesgo:** 70–85%
- **Colapso:** > 85%

### 1.2 Fricción

- **Seguro:** < 3 errores/min
- **Riesgo:** 3–10 errores/min
- **Colapso:** > 10 errores/min

### 1.3 Autonomía

- **Seguro:** 100% decisiones locales
- **Riesgo:** 70–99%
- **Colapso:** < 70%

### 1.4 Integridad informacional

- **Seguro:** discrepancia stock < 2%
- **Riesgo:** 2–10%
- **Colapso:** > 10%

## 1.5 Coherencia estructural

- **Seguro:** 0–2 contradicciones/día
- **Riesgo:** 3–5
- **Colapso:** > 5

## 2 Módulo capaz en pseudocódigo

```
function ciclo_capaz(x):  
    invariantes = evaluar_invariantes(x)  
    estado = clasificar_estado(invariantes)  
  
    if estado == "colapso":  
        aplicar_protocolo_emergencia(x)  
    else if estado == "riesgo":  
        aplicar_protocolo_correccion(x)  
    else:  
        mantener_estabilidad(x)  
  
    return x_actualizado
```

### Evaluación de invariantes

```
function evaluar_invariantes(x):  
    energia = medir_CPU_RAM_temp()  
    friccion = contar_errores_y_ruido()  
    autonomia = medir_decisiones_locales()  
    integridad = comparar_stock_real_vs_digital()  
    coherencia = contar_contradicciones()  
  
    return (energia, friccion, autonomia, integridad, coherencia)
```

### Clasificación del estado

```
function clasificar_estado(inv):  
    if inv.energia > 85% or inv.friccion > 10 or inv.integridad > 10%:  
        return "colapso"  
    if inv.energia > 70% or inv.friccion > 3 or inv.integridad > 2%:  
        return "riesgo"  
    return "seguro"
```

### 3 Evaluación de umbrales

```
function evaluar_umbral(valor, seguro, riesgo, colapso):
    if valor >= colapso:
        return "colapso"
    if valor >= riesgo:
        return "riesgo"
    return "seguro"

estado_energia = evaluar_umbral(CPU, 70, 85, 100)
estado_friccion = evaluar_umbral(errores_min, 3, 10, 999)
estado_autonomia = evaluar_umbral(decisiones_locales, 100, 70, 0)
estado_integridad = evaluar_umbral(discrepancia_stock, 2, 10, 100)
estado_coherencia = evaluar_umbral(contradicciones_dia, 2, 5, 999)

estado_global = max(estado_energia,
                    estado_friccion,
                    estado_autonomia,
                    estado_integridad,
                    estado_coherencia)
```

### 4 Ciclo capaz completo para logística

```
loop cada 5 segundos:
    x = leer_estado_sistema()

    invariantes = evaluar_invariantes(x)
    estado = clasificar_estado(invariantes)

    if estado == "seguro":
        optimizar_flujo_logistico()
    else if estado == "riesgo":
        reducir_friccion()
        restaurar_integridad()
        estabilizar_autonomia()
    else if estado == "colapso":
        detener_procesos_no_esenciales()
        restaurar_energia()
        reconstruir_datos()
        reiniciar_modulos()

    registrar_evento(estado)
end loop
```

## Cierre

GERMEN 02 fija los umbrales del sistema capaz y define su implementación técnica en IA local para logística. Está diseñado para unificarse con GERMEN 00 y GERMEN 01 como tercera pieza del sistema capaz completo.